

Competitive Programming

Section 66: Master+ Mathematics Mixed Contest

Competitive Programming Course

2026-08-20

Table of contents

1	Six problems, one diagnosis discipline	3
2	Contest set	4
3	Escalation pattern	5
4	Proof checkpoints	6
5	Exercises	7
6	Submission checklist	8

1 Six problems, one diagnosis discipline

This section is intentionally not a second lesson. The tools were developed in Sections 64 and 65; the contest tests whether you can recognize them after the surface vocabulary changes.

Before coding a problem, write:

```
exact requested family:  
easier cumulative family or coefficient model:  
inversion / coefficient extraction step:  
target complexity from constraints:  
independent tiny oracle:
```

2 Contest set

Problem	Title
A	Primitive Necklaces
B	Totient Interval
C	Wildcard Rotation
D	Weighted Compositions
E	Black-Box Walks
F	GCD-One Size Spectrum

3 Escalation pattern

A and B test whether the arithmetic identities survived without the lesson's labels. C asks you to derive an index correlation with wildcards. D requires a sparse generating polynomial for each bounded choice. E converts a graph into a scalar sequence before applying algebra. F changes the requested output from one subset count to all subset sizes, forcing a second transform after Mobius inclusion-exclusion.

4 Proof checkpoints

- A: why does “minimal period exactly n ” invert periods dividing n , and why does dividing by n count rotation classes?
- B: why is subtraction of two summatory values valid modulo the prime?
- C: which exact coefficient represents shift r ?
- D: why can degrees above K be discarded permanently?
- E: why must the scalar walk-count sequence satisfy a recurrence of order at most the number of vertices?
- F: why does the Mobius-weighted sum count gcd exactly one for each fixed subset size, and how is the remaining binomial sum one convolution?

5 Exercises

Solve all six as a timed set. During the upsolve, produce a brute generator for each problem before reading the optimized code. The editorial's random tests use independent definitions and are suitable models.

6 Submission checklist

- I derived every reversal/offset instead of copying it from memory.
- All modular subtractions are normalized.
- The NTT is padded to the full linear-convolution length.
- Large integer products and quotient-block endpoints are safe.
- I tested degenerate algebra: order-zero recurrence, all wildcards, $\mathbf{n=1}$, and target sum zero.